

Python Encapsulation: Hiding Data and Enforcing Validation

This comprehensive guide covers how to implement data hiding and validation in Python using encapsulation principles, including public, protected, and private attributes, along with the `@property` decorator for creating getters and setters with validation logic.

Understanding Access Modifiers in Python

Python uses naming conventions with underscores to indicate the intended access level of attributes and methods. Unlike languages like Java or C++, Python doesn't strictly enforce access restrictions but relies on conventions that signal developer intent.

Public Attributes

Public attributes are accessible from anywhere and represent the default behavior in Python.

They use no underscore prefix and are meant to be part of the public API of a class.

```
class Student:
    def __init__(self, name):
        self.name = name # Public attribute

student = Student("Priya")
print(student.name) # Direct access works
```

Public attributes should be used when the data is meant to be accessed and modified freely by external code.

Protected Attributes

Protected attributes use a single underscore prefix (`_attr`) and signal that the attribute is intended for internal use within the class and its subclasses. This is purely a convention—Python doesn't prevent external access, but it communicates to other developers that the attribute should be used with caution.

```
class Car:
    def __init__(self, speed):
        self._speed = 0 # Protected attribute

    def accelerate(self, value):
        self._speed += value

car = Car(0)
car.accelerate(10)
print(car._speed) # Accessible but discouraged
```

Protected members are often used in inheritance hierarchies where subclasses need access to parent class internals.

Private Attributes

Private attributes use a double underscore prefix (`__attr`) and trigger Python's **name mangling** mechanism. When you define `self.__attr`, Python internally renames it to `_ClassName__attr`, making it harder to access accidentally from outside the class.

```

class BankAccount:

    def __init__(self, pin):

        self.__pin = pin # Private attribute


    def verify_pin(self, attempt):

        return self.__pin == attempt


account = BankAccount(1234)

# Direct access fails

try:

    print(account.__pin)

except AttributeError:

    print("Cannot access private attribute!")


# Access via name mangling (not recommended)

print(account._BankAccount__pin) # Works but defeats the purpose

```

Name mangling prevents naming conflicts when subclasses accidentally use the same attribute names as their parent classes. However, it's not true privacy—determined users can still access these attributes through the mangled name.

The @property Decorator

The `@property` decorator is Python's elegant solution for implementing getters and setters without sacrificing the clean syntax of direct attribute access. It allows you to add validation, computation, or logging to attribute access while maintaining a simple interface.^{[13][14][15][16][17]}

Basic Getter Syntax

The `@property` decorator converts a method into a readable attribute:

```
class Employee:

    def __init__(self, name, salary):

        self._name = name

        self._salary = salary


    @property

    def salary(self):

        """Getter method"""

        return self._salary


emp = Employee("Raj", 50000)

print(emp.salary) # Access like an attribute, no parentheses needed
```

The getter method uses `@property` and returns the value of the underlying protected or private attribute.

Setter with Validation

The setter is defined using `@<property_name>.setter` and enables validation before updating the attribute:

```
class Employee:

    def __init__(self, name, salary):

        self._name = name

        self._salary = None

        self.salary = salary # Use setter for initial validation
```

```

@property

def salary(self):

    return self._salary


@salary.setter
def salary(self, value):

    """Setter with validation"""

    if not isinstance(value, (int, float)):

        raise TypeError("Salary must be a number")

    if value < 0:

        raise ValueError("Salary cannot be negative")

    if value < 15000:

        raise ValueError("Minimum salary is $15,000")

    self._salary = value


emp = Employee("Raj", 50000)

emp.salary = 60000 # Validation happens automatically


# Invalid values are rejected
try:

    emp.salary = -5000
except ValueError as e:

    print(e) # Output: Salary cannot be negative

```

The setter method receives the new value as a parameter and can perform any validation before assigning it to the underlying attribute. This ensures data integrity without requiring explicit method calls.

Deleter Method

The deleter is defined using `@<property_name>.deleter` and specifies what happens when an attribute is deleted:

```
class Employee:

    def __init__(self, name, salary):

        self._name = name

        self._salary = salary


    @property
    def salary(self):

        return self._salary


    @salary.setter
    def salary(self, value):

        if value < 0:

            raise ValueError("Salary cannot be negative")

        self._salary = value


    @salary.deleter
    def salary(self):

        """Deleter method"""

        print("Deleting salary...")

        del self._salary


emp = Employee("Raj", 50000)

del emp.salary # Calls the deleter method
```

Deleters are useful when you need to perform cleanup operations or enforce conditions before allowing an attribute to be removed.

Practical Validation Examples

Age and Grade Validation

```
class Student:

    def __init__(self, name, age, grade):

        self._name = name

        self._age = None

        self._grade = None

        self.age = age    # Use setter for validation

        self.grade = grade # Use setter for validation


    @property
    def age(self):

        return self._age


    @age.setter
    def age(self, value):

        if not isinstance(value, int):

            raise TypeError("Age must be an integer")

        if not 5 <= value <= 100:

            raise ValueError("Age must be between 5 and 100")

        self._age = value


    @property
```

```

def grade(self):
    return self._grade

@grade.setter
def grade(self, value):
    if not isinstance(value, (int, float)):
        raise TypeError("Grade must be a number")

    if not 0 <= value <= 100:
        raise ValueError("Grade must be between 0 and 100")

    self._grade = value

student = Student("Priya", 20, 85.5)

student.age = 21

student.grade = 90

# Validation prevents invalid data

try:
    student.age = 150
except ValueError as e:
    print(e) # Age must be between 5 and 100

```

This pattern ensures that objects never contain invalid data by validating in the setter before assignment.

Temperature Validation

```

class Temperature:
    def __init__(self, celsius):
        self._celsius = None

```



```

        self.celsius = celsius # Use setter for validation

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if not isinstance(value, (int, float)):
            raise TypeError("Temperature must be a number")

        if value < -273.15:
            raise ValueError("Cannot be below absolute zero")

        self._celsius = value

    @property
    def fahrenheit(self):
        """Computed property from Celsius"""
        return (self._celsius * 9/5) + 32

    @fahrenheit.setter
    def fahrenheit(self, value):
        """Convert Fahrenheit to Celsius and validate"""
        celsius = (value - 32) * 5/9

        if celsius < -273.15:
            raise ValueError("Cannot be below absolute zero")

        self._celsius = celsius

temp = Temperature(25)

```

```
print(f"{temp.celsius}°C = {temp.fahrenheit}°F")

temp.fahrenheit = 98.6 # Set using Fahrenheit

print(f"{temp.celsius:.1f}°C") # Automatically converted
```

This example demonstrates both validation and computed properties—`fahrenheit` is calculated from `celsius` on-the-fly.

Computed Properties

Computed properties are read-only properties that calculate their value based on other attributes rather than storing the value directly:

```
class Rectangle:

    def __init__(self, width, height):

        self._width = width

        self._height = height

    @property

    def width(self):

        return self._width

    @width.setter

    def width(self, value):

        if value <= 0:

            raise ValueError("Width must be positive")

        self._width = value

    @property
```

```

def height(self):
    return self._height

@height.setter
def height(self, value):
    if value <= 0:
        raise ValueError("Height must be positive")
    self._height = value

@property
def area(self):
    """Computed property – no setter"""
    return self._width * self._height

@property
def perimeter(self):
    """Another computed property"""
    return 2 * (self._width + self._height)

rect = Rectangle(10, 20)
print(rect.area)    # 200
print(rect.perimeter) # 60

rect.width = 15
print(rect.area)    # 300 – automatically updated

```

Computed properties always reflect the current state of the object without needing explicit updates.

Read-Only Properties

To create a read-only property, simply define the getter without a setter:

```
class Person:

    def __init__(self, name, birth_year):

        self._name = name

        self._birth_year = birth_year


    @property

    def name(self):

        return self._name


    @name.setter

    def name(self, value):

        self._name = value


    @property

    def birth_year(self):

        """Read-only – no setter defined"""

        return self._birth_year


person = Person("Amit", 1990)

person.name = "Amit Kumar" # Works


try:

    person.birth_year = 1995 # Fails – no setter

except AttributeError:
```

```
print("Cannot modify birth year!")
```

This pattern is useful for immutable data that should not change after object creation.

Advantages of @property Over Traditional Getters/Setters

The `@property` decorator offers several advantages over traditional getter and setter methods:

Clean Syntax: Properties allow attribute-like access (`obj.attr`) rather than method calls (`obj.get_attr()`), making code more readable and Pythonic.

Backward Compatibility: You can start with a simple public attribute and later add validation by converting it to a property without changing the API.

Computed Values: Properties can compute values on-the-fly without storing redundant data.

Validation: Automatically enforce constraints whenever an attribute is modified.

Encapsulation: Hide implementation details while maintaining a simple interface.

Best Practices

When implementing encapsulation in Python, follow these guidelines:

1. **Make everything as private as necessary:** Use protected (`_`) or private (`__`) attributes for internal implementation details.
2. **Don't overuse private attributes:** The double underscore name mangling can make code harder to read and debug.

3. **Start with public attributes:** Only add property decorators when you need validation or computed values.
4. **Use single underscore for internal APIs:** Protected attributes signal that the attribute may change without notice.
5. **Validate in setters:** Perform all validation checks in setter methods to maintain data integrity.
6. **Create read-only properties judiciously:** Omit the setter for computed or immutable attributes.
7. **Raise appropriate exceptions:** Use `ValueError` for invalid values and `TypeError` for incorrect types.

Summary

Python's encapsulation mechanisms provide a flexible approach to data hiding and validation.

Public attributes (`self.attr`) are accessible everywhere, protected attributes (`self._attr`) signal internal use, and private attributes (`self.__attr`) use name mangling to avoid conflicts in inheritance hierarchies. The `@property` decorator enables clean, Pythonic getters and setters with automatic validation, allowing you to enforce data integrity while maintaining simple attribute-like access.

By combining these tools—access conventions, property decorators, and validation logic—you can write robust, maintainable Python classes that protect data and prevent invalid states.